MODERN C++ :

1. Structured Bindings in c++ (c++ 17 and later)

```cpp
#include <iostream>
#include <tuple>

std::tuple<int, double> func() {
    return std::make_tuple(42, 3.14);
}

int main() {
    auto [a, b] = func();

    std::cout << "a: " << a << std::endl;
    std::cout << "b: " << b << std::endl;

    return 0;
}
```

2. Concurrency with parallel algorithms (c++ 17 and later)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <execution>

int main() {
    std::vector<int> numbers = {1, 5, 3, 2,
4};

    // Use parallel execution policy
    std::for_each(std::execution::par,
numbers.begin(), numbers.end(), [](int& num) {
        num *= 2;
    });

    for (const auto& num : numbers) {
        std::cout << num << " ";
    }

    return 0;
}
```

3. Async Programming using std::async and std::future

```cpp
#include <iostream>
#include <future>
#include <chrono>

// Function to simulate a time-consuming task
int simulateTask() {
    std::cout << "Task started...\n";

std::this_thread::sleep_for(std::chrono::seconds(2));
    std::cout << "Task completed!\n";
    return 42;
}

int main() {
    // Launching an asynchronous task using std::async
    std::future<int> futureResult =
std::async(std::launch::async, simulateTask);

    // Do other work while the task is running
asynchronously

    // Get the result when needed
    int result = futureResult.get();

    std::cout << "Result: " << result << std::endl;

    return 0;
}
```

4. std::filesystem

```cpp
#include <iostream>
#include <filesystem>

namespace fs = std::filesystem;

int main() {
    // Specify the path to a directory
    fs::path directoryPath = "/path/to/your/directory";

    // Check if the path exists
    if (fs::exists(directoryPath)) {
        std::cout << "Directory exists: " << directoryPath << std::endl;

        // Iterate over the contents of the directory
        for (const auto& entry : fs::directory_iterator(directoryPath)) {
            std::cout << entry.path() << std::endl;
        }

        // Create a new directory
        fs::path newDirectory = directoryPath / "NewDirectory";
        if (fs::create_directory(newDirectory)) {
            std::cout << "New directory created: " << newDirectory <<
std::endl;
        } else {
            std::cerr << "Failed to create directory" << std::endl;
        }

        // Rename the new directory
        fs::path renamedDirectory = directoryPath / "RenamedDirectory";
        if (fs::rename(newDirectory, renamedDirectory)) {
            std::cout << "Directory renamed: " << renamedDirectory <<
std::endl;
        } else {
            std::cerr << "Failed to rename directory" << std::endl;
        }
    } else {
        std::cerr << "Directory does not exist: " << directoryPath <<
std::endl;
    }

    return 0;
}
```

5. Auto-typed Variables

```cpp
#include <iostream>
#include <vector>

int main() {
    // Example 1: Auto-typed variable for integer
    auto number = 42;
    std::cout << "Auto-typed variable 'number': "
<< number << " (Type: " << typeid(number).name()
<< ")\n
    // Example 2: Auto-typed variable for a
string
    auto message = "Hello, Auto!";
    std::cout << "Auto-typed variable 'message':
" << message << " (Type: " <<
typeid(message).name() << ")\n";

    // Example 3: Auto-typed variable for a
vector
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    auto iterator = numbers.begin();
    std::cout << "Auto-typed variable 'iterator':
" << *iterator << " (Type: " <<
typeid(iterator).name() << ")\n";

    return 0;
}
```